Authentication & Passwords

Passwords storage

- In the clear…
- Hash of password (done correctly)
  - Doesn't always achieve anything!
  - Makes adversary's job harder
  - Potentially protects users who choose good passwords
- "Salt"-ed hash of password
  - Makes bulk dictionary attacks harder, but no harder to attack a particular password
  - Prevents using 'rainbow tables' generated in advance
- Encrypted passwords? (What attack is this defending against?)
- Centralized server stores password…

Centralized password storage

- Authentication storage node
  - Central server stores password; servers request the password to authenticate user
- Authentication facilitator node
  - Central server stores password; servers send information from user to be authenticated by the central server
- Note that communication with the central server must be authenticated!
  - But can be authenticated using non-password mechanisms

“Do Strong Passwords Accomplish Anything?”

Basic points

- Weak passwords suffice if account locking is used
- Strong passwords are overly burdensome
- Strong passwords do nothing to protect users from most common attacks: phishing or keylogging

- Cost/benefit analysis
  - Are strong passwords worth the effort?

Typical user advice

- Choose strong passwords
- Change passwords frequently
- Don’t write password down (including in your email account…)

Attack taxonomy

- Phishing
- Keylogging
- On-line password guessing for one userID
- On-line password guessing for many userIDs
- Off-line password guessing
- Other
  - Social engineering
  - Password cached on machine

Attack taxonomy

- Phishing/keylogging/other attacks unaffected by password strength
- On-line attacks against one userID are preventable using moderate-strength passwords (next slide)
- Off-line attacks are preventable by using a good protocol

- Main advantage of strong passwords is for on-line attacks against many userIDs

On-line attacks against one user?

- Assumptions
  - 6-digit PIN
  - 24-hour lockdown after 3 failed login attempts
- Number of passwords an attacker can search in 10 years
  - $3 * 365 * 10 \sim 10^4$
- Probability of success
  - $10^4/10^6 = 1\%$

On-line attacks against many users?

- An attack on 10^6 users would likely succeed in breaking in to one of their accounts
  - Account locking has no effect!
- Note that the number of password guesses depends on the number of users
  - N users => 3N password guesses per day (under previous assumptions)

On-line attacks against many users?

- Useful to think in terms of the *credential space* of (userID, password) pairs
  - The adversary breaks-in if it guesses a valid credential
- Say all 25-bit strings are valid userIDs (because userIDs issued sequentially) and 20-bit passwords are used
  - Size of credential space = 2^{45}
  - Number of valid credentials = 2^{20}
  - Success probability per attempt = 2^{-25}
  - Expected attempts to success = 2^{25}

On-line attacks against many users?

- Could decrease attacker's success probability by making the space of legal userIDs more sparse!

- We usually assume userIDs are public (e.g., sent in the clear during login)…
  - …but it would be hard for the attacker to collect very many userIDs

On-line attacks against multiple users?

- Interesting distinction here
  - Users can write down their userIDs
    - Protected against on-line attacks by moderate-strength password and account locking
  - Attacker can get the userID of any particular user
  - Attacker cannot (easily) get the userIDs of many users

- Note that an attacker who *can* easily get many userIDs can perform a DoS attack on the site

On-line attacks against many users?

- Preceding analysis assumes the adversary cannot distinguish an incorrect password guess from an incorrect guess of a userID
  - Be careful in what error messages are returned
  - Be careful of timing attacks

Forgotten passwords

Forgotten passwords

- How to deal with users who forget their passwords?
- Traditional approach: user physically requests password reset (after showing ID, etc.)
- This does not work well over the web…

Forgotten passwords

- *Secret questions* are often used
- These are not very good!
  - 33-39% of answers could be guessed by family members or close friends
  - 20% of users could not remember their own answers!
- Can be improved somewhat using multiple questions, and requiring a threshold of correct answers

Authentication tokens

- RSA SecureID
- PIN-protected memory card
- Cryptographic smartcards
  - Aladdin eTokens
- Smartphone
  - E.g., enter password to device

- Still need a secure protocol!

Biometrics

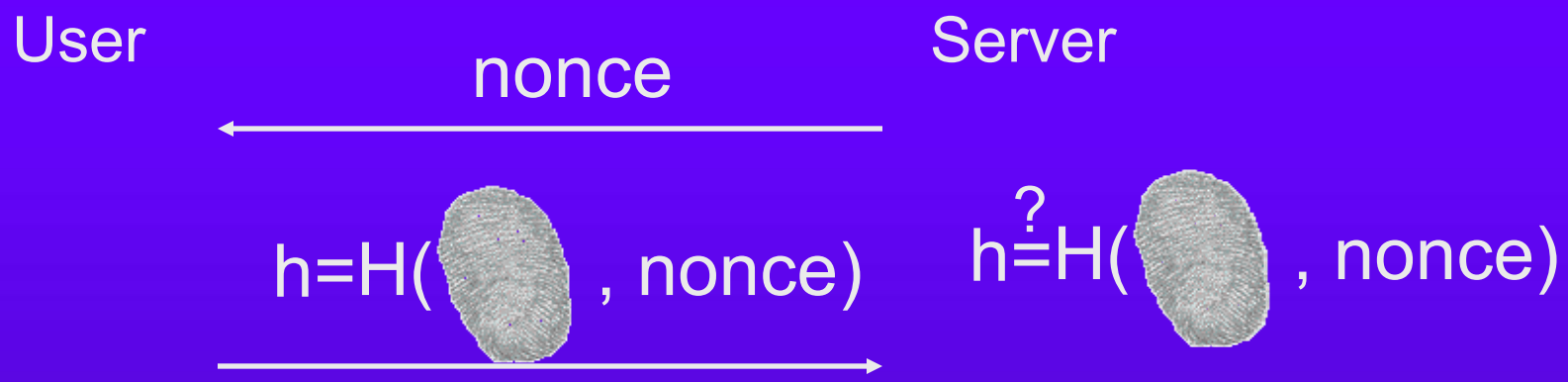
- How much entropy is there?
- How private are they?
- How reproducible are they?
- Revocation?
- Difficult to use securely
  - Errors
  - Non-uniform
  - Still need a secure protocol…

Biometric authentication

- How can you securely authenticate yourself to a remote server using your fingerprint?
- Trivial solution:

Completely vulnerable to eavesdropping!

Better(?) solution



A single-bit difference in the scanned fingerprint results in a failed authentication!

Authentication using biometrics

- There exist techniques for secure authentication using biometric data
  - Resilient to error!
  - Establish random, shared key
- An active research area…

Basic authentication protocols…

- Server stores H(pw); user sends pw
  - If pw is high-entropy, secure against server compromise but not eavesdropping
  - If pw is a password, not secure against server compromise or eavesdropping
- Server stores pw, sends R; user sends $F_{pw}(R)$
  - If pw is a high-entropy key, then this is secure against eavesdropping (but not server compromise)
  - If pw is a password, then this is insecure against an off-line dictionary attack

A public-key protocol

- Server stores pk; user stores sk
- Server sends R; user signs R
  - Using a secure signature scheme…
- Is this secure against eavesdropping/server compromise?
  - What if we had used encryption instead?
- Can we achieve security against eavesdropping and server compromise without public-key crypto?

Lamport's protocol

- Server stores $H^n(pw)$, sends n; user sends $H^{n-1}(pw)$
  - Server updates user's entry…
- Can also add "salt" to hash
  - Server sends salt to user as first flow
  - Allows user to use same password on different sites
  - Can use same password (but different salt) when password "expires"
  - Protects against pre-computation
- Deployed as S/Key